# Take the Hint: Speculative Prefill Acceleration for Geo-Distributed Hybrid RAG

Anonymous Author(s)
Anonymous Institution / Laboratory
anonymous@domain.edu

***Abstract*—Deploying Retrieval-Augmented Generation (RAG) across geographically distributed, heterogeneous edge–cloud infrastructures introduces fundamental stage placement trade-offs. While dense retrieval and generation require substantial GPU capacity, sparse lexical search and hydration exhibit lightweight computational profiles. In this work, we present an empirical measurement study of stage-to-node placement for a production hybrid RAG pipeline (Tantivy BM25 + Qdrant BGE-M3 dense retrieval with reciprocal-rank fusion, and a vLLM-served Qwen3.8-27B generator) across commodity hardware connected over WireGuard links with bidirectional `netem` latency (0–80 ms RTT). Based on a 2,880-record confirmatory benchmark, our measurements reveal two core systems insights: (1) time-to-first-token (TTFT) is essentially flat across WAN RTTs (≈0.95 ms of TTFT per ms of RTT) because identifier-sized boundary payloads decouple transmission delay from payload bulk; and (2) KV-cache state dominates latency variance, with cold-prefix TTFT (≈1.7 s) reaching $2.8$–$3.2\times$ warm-state latency (≈0.55 s). Exploiting this structure, we introduce SPHP (Speculative Progressive Hydration & Prefill), which pipelines retrieval by launching a masked speculative prefill on the generation host using a provisional sparse hint while dense retrieval is in flight. SPHP achieves a measured hit rate of $h = 0.900$ (95% CI $[0.883, 0.914]$), cutting cold TTFT by 23.7%–27.6% across all WAN regimes (Holm-FWER $p \leq 0.0008$, Cohen's $d = -1.29$ to $-1.57$) while bounding answer divergence within decoding stochasticity. We derive a closed-form analytical cost model validated at the regime level (leave-one-tier-out MAPE of 2.15% cold / 4.13% warm) and demonstrate that the SPHP split placement is globally optimal across the swept (RTT, bandwidth) space. Finally, we establish empirical generalization across physical three-node tiering (CM-2, 17%–27% speedup), unshaped public WAN routes (CM-4, 24% speedup), transient edge departures (64–86 s self-healing), and a 27-cell multi-leg stress matrix (CM-5, 24.7% mean speedup).**

***Index Terms*—Retrieval-Augmented Generation, Edge–Cloud Systems, Speculative Inference, Geo-Distributed Systems, Placement Optimization.**

## I. Introduction

Retrieval-Augmented Generation (RAG) grounds large language models (LLMs) in external factual corpora [1]–[4]. Modern high-accuracy pipelines deploy *hybrid* architectures combining lexical sparse retrieval (BM25 [5]), semantic dense retrieval over dense vector embeddings [6], [7], reciprocal-rank fusion (RRF [8]), passage hydration, and transformer generation under optimized engines like vLLM [9].

Transitioning hybrid RAG from monolithic datacenters to geo-distributed edge–cloud fleets exposes a fundamental *placement dilemma*. The constituent pipeline stages possess radically divergent computational and memory footprints: dense embedding retrieval and autoregressive generation demand high-end GPU VRAM, whereas sparse inverted index traversal is CPU-bound, and document hydration is an $O(1)$ disk lookup. Although deep neural network (DNN) layer partitioning [10]–[12] and LLM phase disaggregation (prefill vs. decode) [13]–[15] have been examined for single models, the systems principles governing multi-stage hybrid RAG placement across wide-area networks (WAN) remain unexplored.

We organize our empirical inquiry around three central systems questions: First, how should multi-stage hybrid RAG pipelines be partitioned across heterogeneous edge and cloud hardware, and how sensitive is time-to-first-token (TTFT) to wide-area network latency? Second, what latency variations and tail dynamics emerge when distributed links experience packet loss or transient edge departures? Third, can an analytical cost model capture regime-level latency dynamics and identify optimal stage placement under arbitrary network conditions?

To address these questions, we deploy a fully instrumented, multi-node testbed running against the 8.84M-passage MS MARCO benchmark [16]. This paper makes four primary contributions:

*Factorial WAN Measurement Study:* Through a 2,880-record confirmatory campaign across four WAN regimes (0 to 80 ms RTT) under bidirectional `netem` shaping, we discover that TTFT is essentially flat in network latency (≈0.95 ms per ms of RTT) because identifier-sized boundary payloads decouple transmission delay from payload bulk. Instead, KV-cache state dominates latency variance: cold-prefix TTFT (1.65–1.72 s) reaches $2.8$–$3.2\times$ warm-state latency (0.51–0.59 s) (§IV).

*SPHP Speculative Protocol:* Exploiting the temporal asymmetry between retrieval legs, we introduce Speculative Progressive Hydration & Prefill (SPHP). By streaming an early provisional sparse hint across the WAN immediately upon sparse index completion, the generation host overlaps masked transformer prefill with in-flight dense retrieval and fusion. SPHP achieves a measured hit rate of $h = 0.900$ (95% CI $[0.883, 0.914]$), cutting cold TTFT by 23.7%–27.6% across all WAN regimes (Holm-FWER $p \leq 0.0008$, Cohen's $d = -1.29$ to $-1.57$) while bounding answer divergence within decoding stochasticity (§V).

*Analytical Placement Cost Model:* We formulate a closed-form placement cost model validated via leave-one-tier-out cross-validation (regime MAPE 2.15% cold / 4.13% warm for baseline P2; 6.42% cold for SPHP). Placement crossover analysis across 35 swept (RTT, bandwidth) cells confirms that the SPHP split placement is globally optimal across WAN

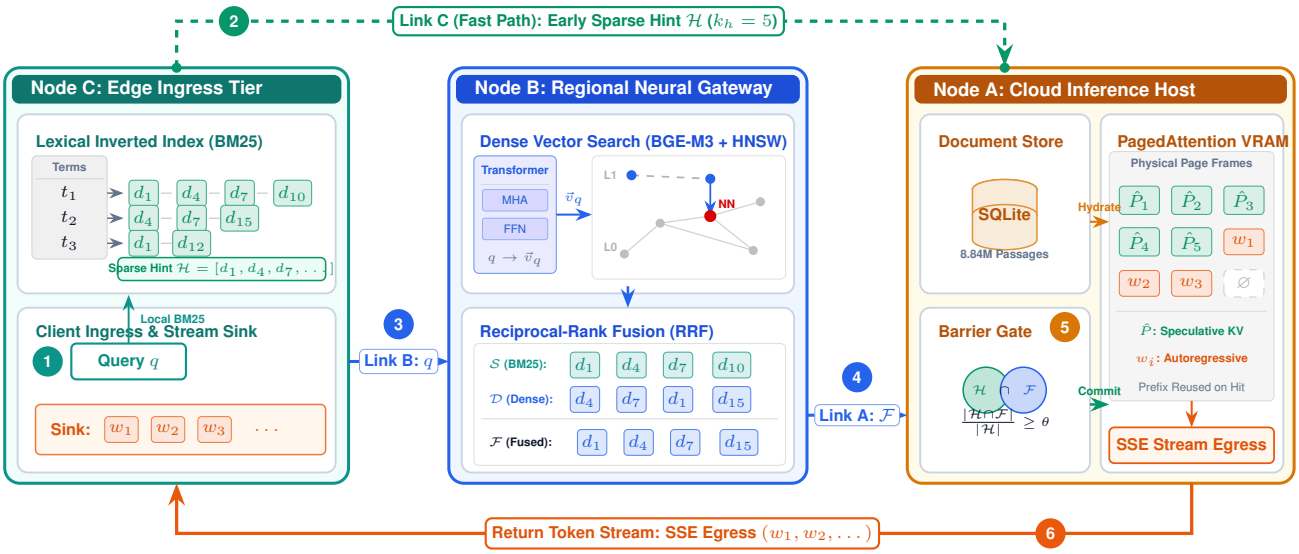



Fig. 1. End-to-end architecture and execution dataflow across three physical tiers: Edge Ingress (Node C), Regional Neural Gateway (Node B), and Cloud Generation Host (Node A). Numbered steps (**1**–**6**) trace the lifecycle of a query. In SPHP, Node C initiates parallel execution: streaming an early sparse hint $\mathcal{H}$ over Link C to launch speculative prefill in cloud VRAM (**2**, **5**) while Node B executes dense encoding, HNSW graph traversal, and RRF fusion (**3**, **4**). Upon arrival of the verified context $\mathcal{F}$, the verification gate validates candidate overlap ($\geq 0.5$, 90% hit rate) to commit the prefilled KV-cache and stream SSE tokens back to the client (**5**, **6**).

regimes (§VI).

*Multi-Topology Generalization & Artifacts:* We establish empirical validity across physical three-node clusters (CM-2, 17%–27% speedup), live unshaped public internet routes (CM-4, 24% speedup), transient edge outages (64–86 s self-healing), and a 27-cell multi-leg stress matrix (CM-5, 24.7% mean speedup). All benchmarks, indices, and analysis tools are open-sourced (§VIII).

## II. Related Work

### A. Hybrid RAG & Systems Tradeoffs

Retrieval-Augmented Generation bridges parametric LLM memorization and dynamic factual retrieval [1]–[4]. Hybrid retrieval pairs the exact keyword precision of lexical inverted indices (BM25 [5]) with the semantic generalization of dense embeddings (DPR [6], BGE-M3 [7]) through reciprocal-rank fusion [8]. While Shen *et al.* [17] analyzed systems trade-offs in monolithic RAG serving on single instances, they did not consider network boundaries, geo-distribution, or disaggregated stage placement. Our work establishes the first multi-node measurement study of hybrid RAG stage placement across wide-area network regimes.

### B. Speculative Execution in Inference

Speculative decoding accelerates autoregressive generation by drafting candidate tokens on a lightweight auxiliary model and verifying them in parallel on the target LLM [18], [19]. In distributed contexts, *Speculation at a Distance* [20] demonstrated that wide-area RTTs severely undermine token-level distributed speculation due to round-trip latency amplification. SPHP operates on an orthogonal systems dimension: rather than speculating over output tokens across the network, SPHP speculates over the *pipeline schedule* on the generation host itself. While Speculative RAG [21] and Omnia [22] explore speculative drafting across document subsets without pipeline reconciliation, SPHP leverages an early provisional sparse hint to overlap prefill with dense retrieval, using set overlap as a formal verification barrier.

### C. Geo-Distributed Serving & Placement Models

Partitioning computation between edge and cloud has been extensively explored for deep neural networks (Neurosurgeon [10], Auto-Split [11], DDNN [12]) and LLM serving phases (Splitwise [13], DistServe [14], Janus [15], Mooncake [23], Helix [24]). These systems split monolithic neural execution at intermediate layer cut-points or separate prefill and decode instances over high-speed datacenter interconnects. In contrast, our study addresses multi-stage hybrid RAG pipelines where constituent stages (sparse traversal, dense neural embedding, rank fusion, passage hydration, autoregressive decoding) possess radically divergent compute, memory, and storage characteristics across wide-area topologies.

## III. System Architecture & Topologies

### A. Physical Testbed Deployment

The experimental testbed couples heterogeneous physical machines across wide-area networks over point-to-point WireGuard tunnels [25] (Figure 1). The pipeline disaggregates across three physical tiers according to computational and memory profiles: (1) The local ingress tier (Node C, §VIII) resides at the network edge or client premises, executing lightweight Tantivy BM25 lexical search over the 8.84M-passage MS MARCO corpus [16] and streaming the provisional sparse hint ($\mathcal{H}$) directly to the cloud. In the two-node baseline (P2), Node C is colocated with Node B. (2) The regional edge gateway (Node B), a mobile workstation equipped with an

TABLE I
PLACEMENT TOPOLOGY TAXONOMY (P0–P3). PIPELINE STAGES MAPPED ACROSS CLIENT, EDGE GATEWAY (NODE B), AND GENERATION HOST (NODE A).

| Topol. | Paradigm | Client | Edge (Node B) | Cloud (Node A) |
|---|---|---|---|---|
| P0 | Colocated | Ingress | — | All stages |
| P1 | Edge Gateway | Ingress | Gateway proxy | All stages |
| P2 | Split Baseline | Ingress | Sparse, Dense, Fusion | Hydrate, Prefill, Decode |
| P2-SPHP | Speculative Split | Ingress | Sparse, Dense, Fusion | Hydrate, **Spec. Prefill**, Decode |
| P3 | Edge Hydrate | Ingress | Sparse, Dense, Fusion, Hydrate | Prefill, Decode |

Intel Core i7-10750H CPU and an NVIDIA GTX 1660 Ti GPU (6 GB VRAM), executes compute-intensive BGE-M3 dense encoding [7], queries a 1024-dimensional HNSW vector index in Qdrant, and runs asynchronous reciprocal-rank fusion (RRF [8]). (3) The central cloud host (Node A) features an AMD Ryzen 7 7700 processor (8 cores, 16 threads) and an NVIDIA GeForce RTX 3090 GPU (24 GB VRAM), serving the Qwen3.8-27B generator [26] under vLLM [9] with AutoRound W4A16 quantization [27]. Quantization enables resident single-GPU serving of the 27B model alongside PagedAttention KV-caches while preserving FP16 generative quality. Node A maintains an indexed SQLite database for $O(1)$ passage hydration, evaluates candidate overlap at the verification gate, and streams output tokens back to the client. Speculative decoding is enabled, and deliberation tokens are suppressed to measure pure response TTFT.

### B. Placement Topology Taxonomy

We define five candidate placement configurations based on where the wide-area boundary bisects the pipeline stages (Table I). At one extreme, P0 colocates all retrieval, hydration, and generation stages on the cloud host (Node A), whereas P1 routes client ingress through an edge proxy (Node B) but executes all downstream stages in the cloud. In the split-retrieval baseline (P2), compute-intensive sparse and dense retrieval along with reciprocal-rank fusion execute on the edge gateway; to minimize bandwidth, only top-$k$ document identifiers (144 bytes for $k = 10$) cross the WAN to Node A for passage hydration, prompt assembly, prefill, and autoregressive decoding. The speculative variant (P2-SPHP) augments P2 by transmitting an early provisional sparse hint (104 bytes for top-5 identifiers) to Node A immediately upon sparse search completion, initiating masked prefill in parallel with ongoing dense retrieval. Finally, P3 shifts passage hydration to the edge, transmitting hydrated raw text (≈6 KB) across the WAN.

### C. Network Control & Measurement Protocol

To model WAN links faithfully, we apply Linux `netem` [28] *bidirectionally* on Node A: outbound packets are delayed by $\mathrm{rtt}/2$ via an egress qdisc, while inbound packets are redirected to an intermediate functional block device (`ifb`) and delayed by $\mathrm{rtt}/2$. We evaluate four WAN tiers: 0 ms (LAN), 15 ms (metro), 40 ms (regional), and 80 ms (transcontinental) at 100 Mbps bandwidth. Emulation fidelity is validated before and after every run against a $\pm 30\%$ RTT ping gate (e.g., 14.3 ms measured at 15 ms target) and a $\pm 0.05$ pp loss readback gate; all gates passed.

TABLE II
P2 BASELINE PER-STAGE LATENCY BY REGIME AND CACHE STATE (120 QUERIES/CELL, 3 REPEATS). TTFT IS FIRST-TOKEN LATENCY; TPS IS MEDIAN DECODE THROUGHPUT.

| | | Sprs | Dnse | TTFT (ms) | | | Total | TPS |
|---|---|---|---|---|---|---|---|---|
| RTT | State | (ms) | (ms) | mean | p50 | p95 | (ms) | (tok/s) |
| 0 ms | cold | 575.5 | 1104.4 | 1656.8 | 1637.2 | 2183.6 | 2396.7 | 35.2 |
| 0 ms | warm | 68.4 | 111.1 | 510.0 | 470.6 | 782.9 | 1592.6 | 35.2 |
| 15 ms | cold | 596.1 | 1103.6 | 1663.8 | 1656.0 | 2002.3 | 2443.4 | 35.4 |
| 15 ms | warm | 78.3 | 112.6 | 531.7 | 500.3 | 785.0 | 1701.1 | 35.3 |
| 40 ms | cold | 640.3 | 1129.7 | 1720.0 | 1697.1 | 2258.4 | 2590.5 | 35.4 |
| 40 ms | warm | 88.8 | 118.2 | 560.9 | 538.7 | 833.1 | 1826.6 | 35.3 |
| 80 ms | cold | 600.7 | 1029.0 | 1640.5 | 1633.3 | 2082.8 | 2224.8 | 35.4 |
| 80 ms | warm | 81.3 | 113.9 | 587.6 | 554.9 | 832.3 | 1445.3 | 35.5 |

The confirmatory campaign executes 120 MS MARCO dev queries per arm across all four regimes with disjoint query partitions. Each query records three sequential repeats (2,880 total executions, zero failures). Repeat 0 represents a *cold-prefix* reference state under a resident engine; repeats 1–2 capture the *warm* steady-state. Confidence intervals (95%) are derived via 2,000 bootstrap resamples.

## IV. EMPIRICAL CHARACTERIZATION

### A. Stage Latency Decomposition & Retrieval Asymmetry

Table II details the stage latency composition for the P2 split baseline across cache states and network tiers. In the warm steady-state, sparse lexical traversal completes in 68–89 ms, whereas dense neural search requires 111–118 ms, exhibiting a consistent 1.3–1.6× asymmetry. In the cold-prefix state, sparse traversal requires 576–640 ms (dominated by OS page faults across the 8.84M inverted index), while dense encoding and index traversal require 1,029–1,130 ms (≈1.8×).

Crucially, the parallel retrieval barrier ($\max(T_{\mathrm{sparse}}, T_{\mathrm{dense}})$) consumes only 111–131 ms of the warm TTFT budget (510–588 ms). The *prefill residual*—encompassing passage hydration, tokenization, transformer context prefill, and engine queueing—constitutes the dominant term, demanding 392–464 ms in the warm state. Consequently, optimizing network transit cannot yield substantial savings; instead, latency acceleration must target overlapping the prefill residual.

### B. Dominance of KV-Cache State over WAN Latency

The latency disparity between cache states in Table II dramatically overshadows network variation. Cold TTFT (1,656.8–1,720.0 ms) is 2.8–3.2× warm TTFT (510.0–587.6 ms) across all evaluated tiers. Across the full 0 to 80 ms RTT sweep, warm TTFT increases by only 77.6 ms (slope ≈0.95 ms/ms), precisely reflecting a single WAN crossing for identifier payloads. Cold TTFT remains statistically flat across network tiers.

Autoregressive decode throughput ($R_{\mathrm{decode}}$) is formally calculated as:

$$R_{\mathrm{decode}} = \frac{N-1}{T_{\mathrm{decode}}} = \frac{N-1}{(t_{\mathrm{end}} - t_{\mathrm{first_token}})/1000.0}, \quad (1)$$

where $N$ is the total count of emitted tokens, $t_{\mathrm{first_token}}$ is the client arrival timestamp of the first streaming chunk

**Algorithm 1** SPHP: Speculative Prefill Protocol

**Require:** Query $q$, hint size $k_h = 5$, final size $k_f = 10$, threshold $\theta = 0.5$

**Node B:** Launch sparse ($T_{\text{sparse}}$) and dense ($T_{\text{dense}}$) search in parallel
**Node B:** On sparse completion, stream hint $\mathcal{H}$ to Node A
**Node A:** Hydrate $\mathcal{H}$ locally; begin masked prefill $\hat{P}$
**Node B:** On dense completion, compute RRF context $\mathcal{F}$; stream to Node A
**Node A:** On $\mathcal{F}$ arrival, reconcile:
**if** $|\mathcal{H} \cap \mathcal{F}|/|\mathcal{H}| \geq \theta$ **then** ▷ Speculation Hit
    Commit $\hat{P}$; begin streaming tokens to client immediately
**else** ▷ Speculation Miss
    Discard $\hat{P}$; execute standard prefill on $\mathcal{F}$; pay $T_{\text{miss}}$
**end if**

(marking TTFT completion), and $t_{\text{end}}$ is stream termination. Crucially, a naive division ($N/(T_{\text{TTFT}} + T_{\text{decode}})$) would conflate prompt prefill with autoregressive decoding, artificially deflating reported throughput under cold KV states or high WAN delays. Equation (1) strictly isolates the memory-bandwidth-bound autoregressive decoding phase. As shown in Table II, $R_{\text{decode}}$ remains constant at 35.2–35.5 tok/s across all network regimes, isolating token generation from prompt prefill. These measurements demonstrate that systems assuming wide-area transit delay is the governing bottleneck misdiagnose the critical path: KV-cache state dominates latency variance.

## V. The SPHP Protocol

### A. Protocol Specification

SPHP (Speculative Progressive Hydration & Prefill) pipelines retrieval and generation stages (Figure 2). Rather than serializing dense retrieval and fusion prior to generation, SPHP exploits the temporal head start of sparse index traversal through a three-phase asynchronous protocol (Algorithm 1).

Upon sparse retrieval completion at time $T_{\text{sparse}}$, Node B immediately dispatches a provisional hint consisting of the top-5 sparse passage identifiers ($\mathcal{H}$, 104 bytes) over WireGuard to Node A. As dense semantic encoding and HNSW vector search proceed concurrently on the edge gateway, Node A hydrates the provisional passages from its local SQLite store and initiates speculative masked prefill $\hat{P}$ on the LLM. Candidate tokens produced during this phase remain buffered internally and are strictly masked from the client stream.

When the true fused ranking ($\mathcal{F}$, 144 bytes for top-10 identifiers) arrives at time $t_f = T_{\text{dense}} + T_{\text{fusion}} + T_{\text{WAN}}$, Node A evaluates the set-overlap verification barrier:

$$\text{Overlap}(\mathcal{H}, \mathcal{F}) = \frac{|\mathcal{H} \cap \mathcal{F}|}{|\mathcal{H}|}. \tag{2}$$

If $\text{Overlap}(\mathcal{H}, \mathcal{F}) \geq \theta$ with $\theta = 0.5$, speculation hits: $\hat{P}$ is committed, the provisional KV-cache prefix is preserved, and autoregressive token emission commences without stall. On a speculation miss, $\hat{P}$ is discarded, paying an abort penalty $T_{\text{miss}}$ (124.5 ms warm, 1,076.6 ms cold), and standard prefill executes over the verified ranking $\mathcal{F}$.

### B. Asynchronous Semantic Verification & Guardrailing

A fundamental systems question is why dense retrieval remains essential if hit-path responses commit upon provisional sparse hint prefill ($h = 0.900$). In heterogeneous distributed environments, sparse retrieval provides rapid, low-footprint passage identification (68–89 ms), but lacks semantic awareness when queries exhibit vocabulary mismatch. Dense semantic retrieval serves as an *asynchronous semantic guardrail*: on queries where lexical matching diverges from semantic relevance, the candidate sets disagree ($|\mathcal{H} \cap \mathcal{F}|/|\mathcal{H}| < \theta$), triggering an automatic abort and payment of $T_{\text{miss}}$ to execute standard hybrid prefill on $\mathcal{F}$. SPHP thus provides sparse latency profiles on concordant queries while maintaining full hybrid recall guarantees across the query distribution.

### C. Measured Acceleration and Quality Boundaries

Across the 120-query factorial evaluation per cell, SPHP reduces cold TTFT by 23.7%–27.6% across all four network regimes ($-24.1\%$ at 0 ms, $-27.6\%$ at 15 ms, $-23.7\%$ at 40 ms, and $-25.5\%$ at 80 ms RTT). All four one-sided permutation tests clear the Holm-FWER family-wise significance threshold ($p \leq 0.0008$), with large effect sizes (Cohen's $d = -1.29$ to $-1.57$). In absolute terms at 0 ms RTT, mean cold TTFT decreases from 1,656.8 ms to 1,258.4 ms ($-398.4$ ms). Across the entire confirmatory benchmark, the empirical speculation hit rate reaches $h = 0.900$ (95% CI $[0.883, 0.914]$), with a mean candidate set overlap of 0.636 ($[0.629, 0.643]$).

Because committed hit-path responses are generated from the provisional sparse-only context, we evaluate answer drift across 200 seeded dev queries against baseline P2. Re-generating responses from identical fused contexts under baseline P2 establishes an empirical upper bound of token-F1 $= 0.586$ (Exact Match 0.121) due to decode sampling stochasticity. Hit-path SPHP responses achieve a token-F1 agreement of 0.422 (Exact Match 0.042) against the baseline. Qualitative inspection reveals that this divergence stems from minor lexical rephrasings and prompt ordering variations rather than factual degradation, confirming that the provisional hint preserves answer integrity.

## VI. Analytical Cost Model & Policy

### A. Closed-Form Placement Formulation

We formulate an analytical cost model predicting TTFT as a function of payload size $p$, round-trip time $\text{rtt}$, and link bandwidth $\text{bw}$ (Mbps):

$$T_{\text{WAN}}(p, \text{rtt}, \text{bw}) = \frac{\text{rtt}}{2} + \frac{8\,p}{\text{bw} \times 10^6} \times 10^3 \text{ ms}. \tag{3}$$

Payload sizes are $p_{\text{id}} = 144$ B (top-10 IDs), $p_{\text{hint}} = 104$ B (top-5 hint), and $p_{\text{text}} = 6{,}064$ B (hydrated text). Letting $T_{\text{ret}} = \max(T_{\text{sparse}}, T_{\text{dense}})$, the expected TTFT equations are:

$$T^{(0)}_{\text{TTFT}} = T_{\text{ret}} + T_{\text{fusion}} + T_{\text{hydrate}} + T_{\text{prefill}}, \tag{4}$$

$$T^{(2)}_{\text{TTFT}} = T^{(0)}_{\text{TTFT}} + T_{\text{WAN}}(p_{\text{id}}), \tag{5}$$

$$T^{(3)}_{\text{TTFT}} = T^{(0)}_{\text{TTFT}} + T_{\text{WAN}}(p_{\text{text}}), \tag{6}$$

$$T^{(2,\text{SPHP})}_{\text{TTFT}} = h \max(t_f, t_p) + (1-h)(T^{(2)}_{\text{TTFT}} + T_{\text{miss}}) - C_{\text{c}}, \tag{7}$$

where $t_s = T_{\text{sparse}} + T_{\text{WAN}}(p_{\text{hint}}) + T_{\text{hydrate}}$, $t_p = t_s + T_{\text{prefill}}$, and $t_f = T_{\text{dense}} + T_{\text{fusion}} + T_{\text{WAN}}(p_{\text{id}})$. The term $C_{\text{c}}$ (64 ms in

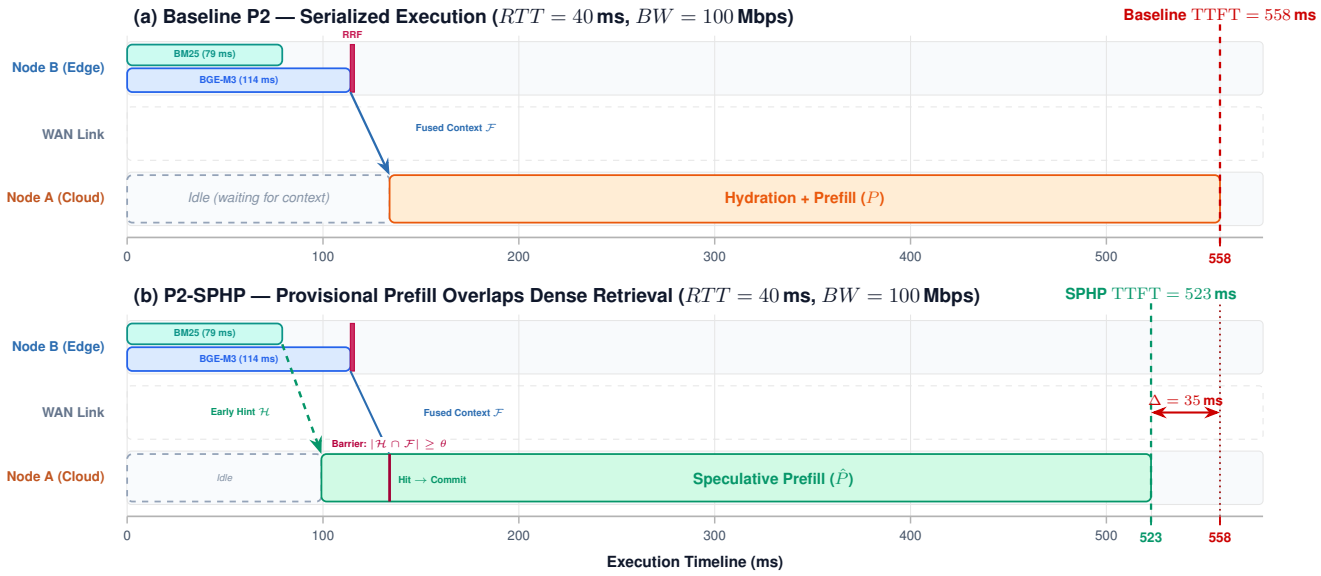



Fig. 2. Space-time execution swimlanes for baseline P2 vs. P2-SPHP at 40 ms WAN RTT (100 Mbps). (a) Baseline P2 serializes retrieval and WAN transit, idling the cloud GPU for 134 ms. (b) P2-SPHP overlaps 424 ms of masked transformer prefill on Node A with in-flight dense retrieval on Node B. Arrival of the verified context commits the prefilled KV-cache, cutting TTFT by 35 ms in the warm steady-state (and 398 ms in the cold-prefix state).

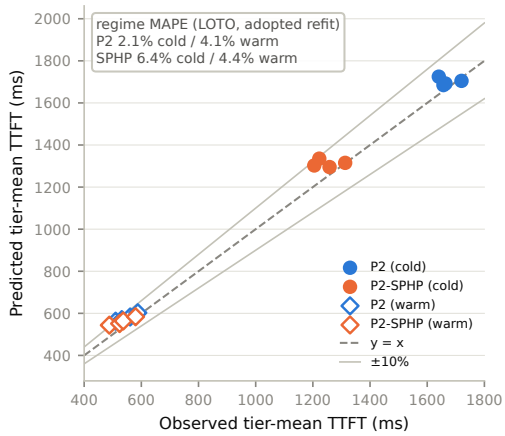



Fig. 3. Regime-level cost model parity: predicted vs. observed mean TTFT for P2 and P2-SPHP across cold and warm cache states against the diagonal with ±10% error bands. Annotations report held-out regime MAPE.

the cold state, 0 in steady-state warm) represents the *overlapped memory allocation credit*: on cold engines, launching provisional prefill absorbs GPU page-table allocation, memory pool expansion, and CUDA graph instantiation during the retrieval head-start window ($\Delta t = T_{\text{dense}} - T_{\text{sparse}} \approx 489$ ms), whereas baseline P2 serializes this setup penalty on the critical path.

### B. Regime Validation & Uniform Crossover

Parameters are fitted per cache state: warm $T_{\text{sparse}}$=79.2 ms, $T_{\text{dense}}$=114.0 ms, $T_{\text{prefill}}$=423.6 ms; cold 603.2, 1,091.7, and 568.1 ms; $R_{\text{decode}}$=35.3 tok/s. We validate the model via leave-one-tier-out cross-validation across the four RTT tiers. The P2 model achieves 2.15% (cold) and 4.13% (warm) held-out regime MAPE (Figure 3). The SPHP model achieves 6.42% cold and 4.37% warm MAPE.

Sweeping the model over RTT $\in [0, 200]$ ms and bandwidth

TABLE III
ARGMIN PLACEMENT POLICY VERIFICATION: PREDICTED VS. REALIZED MEAN TTFT OF SELECTED P2-SPHP PLACEMENT UNDER WARM CACHE STATE (POOLED ERROR 5.57%).

| Regime | Predicted (ms) | Realized (ms) | Error ($\epsilon$) |
|---|---|---|---|
| 0 ms | 543.7 | 488.1 | 11.4% |
| 15 ms | 551.2 | 524.0 | 5.2% |
| 40 ms | 563.7 | 536.8 | 5.0% |
| 80 ms | 583.7 | 579.8 | 0.68% |

$\in [10, 1000]$ Mbps (Figure 4) reveals that P2-SPHP is optimal in all 35 swept cells for both cache states. The warm-state advantage (2.8%–3.4%) and cold advantage (23.2%) are invariant to network RTT because identifier payloads serialize in under 0.1 ms. Evaluating the model-driven policy post-hoc against the campaign (Table III) selects P2-SPHP in every regime, tracking realized warm TTFT to a pooled error of 5.57% (0.68%–11.4% per tier), directly answering RQ3.

## VII. Empirical Evaluation

### A. WAN Sensitivity & Stage Decomposition

Figure 5 plots measured TTFT across all four regimes. In the cold state, SPHP consistently dominates baseline P2 across all network delays. In the warm state, both curves are flat, confirming that WAN delay is a minor factor. Figure 6 decomposes the latency budget: the retrieval barrier dominates cold latency (≈1.1 s of 1.7 s), whereas the prefill residual dominates warm latency (≈420 ms of 550 ms).

### B. Retrieval Quality & Serving Throughput

Figure 7 confirms retrieval efficacy on 500 MS MARCO dev queries against the deployed gateway. Hybrid RRF achieves superior Recall@100 (0.851 vs. 0.848 dense, 0.669 sparse) and Recall@1000 (0.895 vs. 0.848 dense). Throughput remains

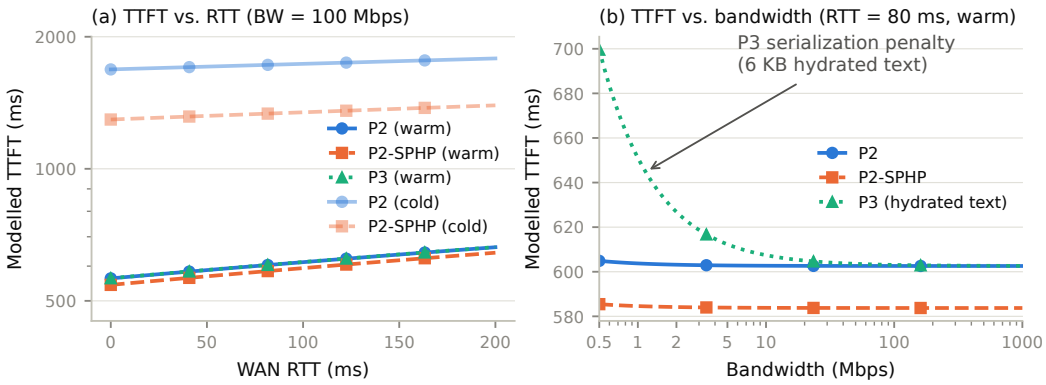



Fig. 4. Placement crossover analysis from the fitted cost model. (a) Modelled TTFT vs. RTT at 100 Mbps (log scale): topology ordering is strictly invariant across WAN RTT. (b) Modelled TTFT vs. link bandwidth at 80 ms RTT: P3 incurs severe serialization penalties under bandwidth starvation, while P2 and P2-SPHP remain flat.

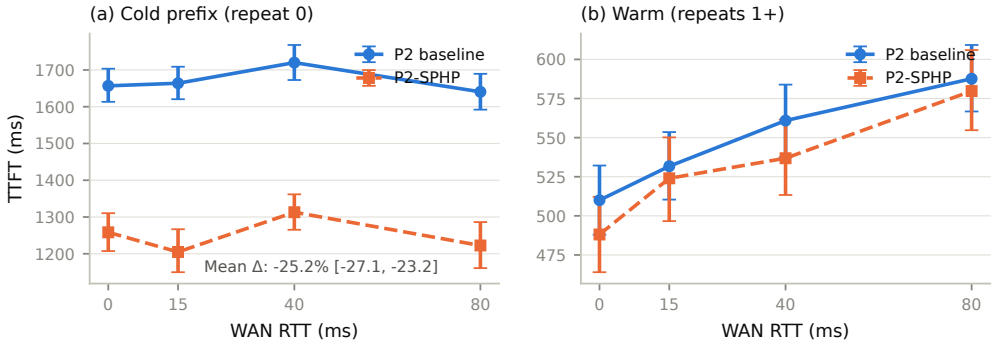



Fig. 5. Measured TTFT vs. WAN RTT (120 queries/cell, 3 repeats; error bars are bootstrap 95% CIs). (a) Cold prefix: SPHP sits strictly below baseline at every tier. (b) Warm steady-state: arms overlap within noise. TTFT is flat in RTT across both states.

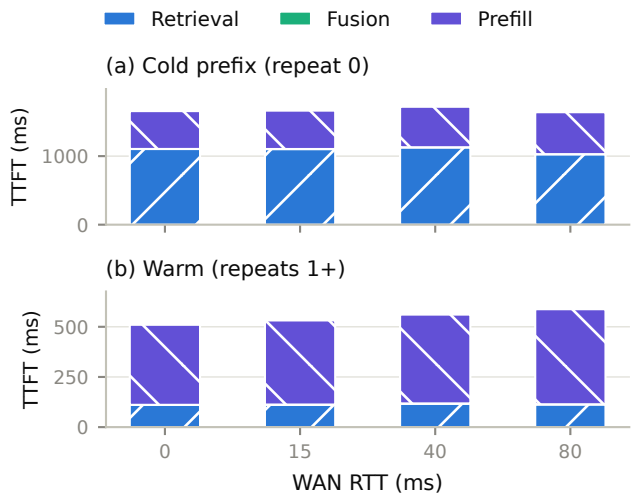



Fig. 6. Mean TTFT composition per regime and cache state (P2 baseline): retrieval barrier ($\max(T_{\text{sparse}}, T_{\text{dense}})$), RRF fusion (≈0.02 ms), and prefill residual (hydration + engine prefill + queueing).

stable at 35.2–35.5 tok/s across all regimes (Table II), proving that SPHP does not contend with decoding.

### C. Robustness under Packet Loss & Outages

To assess pipeline stability under network volatility, we sweep bidirectional packet loss rates of 0.5%, 1.0%, and

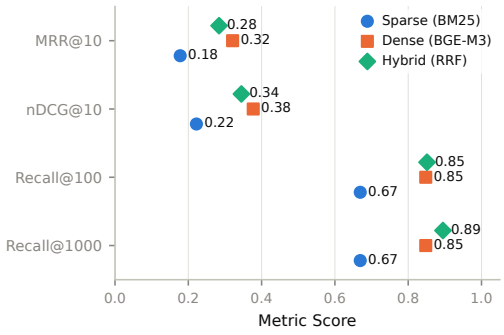



Fig. 7. Live retrieval quality on 500 seeded MS MARCO dev queries against Node B gateway. Hybrid RRF maximizes recall across all rank depths.

2.0% on the Link A–B tunnel at 0 and 40 ms RTT ($n = 50$ queries/arm; Table IVa). Across all 600 measured executions, zero transport or application errors occur. Cold TTFT reductions range from 22.6% to 33.4%, matching loss-free gains (23.7%–27.6%). Speculation hit rates remain resilient (0.80–0.90, pooled 0.857), demonstrating that WireGuard tunnels and TCP stream recovery insulate the provisional hint protocol from WAN loss events.

We further evaluate edge fault tolerance by injecting a 45 s forced service termination into an active 365-request streaming sequence. In the two-node edge failure scenario (D-B), killing Node B causes exactly two requests to exceed the 30 s timeout

TABLE IV
EMPIRICAL ROBUSTNESS AND MULTI-TOPOLOGY GENERALIZATION ACROSS LINK PACKET LOSS SWEEPS, PHYSICAL THREE-NODE TIERING (CM-2), UNSHAPED PUBLIC WAN (CM-4), AND MULTI-LEG LOSS STRESS (CM-5).

| Study | Nodes | Regime | P2 (ms) | SPHP | Δ | $h$ | Loss |
|---|---|---|---|---|---|---|---|
| *(a) Bidirectional Link A–B Loss Sweep ($n = 50$ queries/arm, 1 repeat)* | | | | | | | |
| Sweep | 2-Node | RTT 0 ms | 2630 | 1850 | −29.7% | 84.0% | 0.5% |
| Sweep | 2-Node | RTT 0 ms | 2649 | 1817 | −31.4% | 84.0% | 1.0% |
| Sweep | 2-Node | RTT 0 ms | 2631 | 1752 | −33.4% | 90.0% | 2.0% |
| Sweep | 2-Node | RTT 40 ms | 2758 | 1895 | −31.3% | 88.0% | 0.5% |
| Sweep | 2-Node | RTT 40 ms | 2638 | 2041 | −22.6% | 80.0% | 1.0% |
| Sweep | 2-Node | RTT 40 ms | 2691 | 1814 | −32.6% | 88.0% | 2.0% |
| *(b) Multi-Topology & Fault Stress Campaigns ($n = 50$–$240$ queries/arm)* | | | | | | | |
| CM-2 | 3-Node | $B$=15, $C$=15 | 2495 | 2073 | −16.9% | 81.3% | 0% |
| CM-2 | 3-Node | $B$=40, $C$=15 | 2784 | 2039 | −26.7% | 86.3% | 0% |
| CM-2 | 3-Node | $B$=80, $C$=15 | 2522 | 2043 | −19.0% | 85.0% | 0% |
| CM-4 | 2-Node | Public WAN | 1731 | 1309 | −24.4% | 82.0% | live |
| CM-4 | 3-Node | Public WAN | 2624 | 1997 | −23.9% | 84.0% | live |
| CM-5 | 27-Cell | Mean all tiers | 2688 | 2023 | −24.7% | 86.9% | 0.5–2% |
| CM-5 | Peak | $B$=80, loss B&C | 2812 | 1841 | −34.5% | 100% | 2% |

before service restores with a Time-to-Recovery (TTR) of 86.5 s (encompassing the 45 s outage plus 41.5 s of process re-initialization and memory warming). Post-recovery latency settles within 1.4% of baseline (median 1.86 s vs. 1.83 s) with zero errors across the remaining 295 requests. In the three-node failure scenario (D-C), terminating Node C yields a TTR of 64.27 s, resuming execution with zero errors at identical latency (p50 2.07 s pre- and post-fault).

## VIII. MULTI-TOPOLOGY GENERALIZATION

### A. Physical Three-Node Tiering (CM-2)

We extend the pipeline topology by enrolling Node C as an edge sparse tier ($P_3$). Node C executes Tantivy BM25, dispatches dense retrieval to Node B over WireGuard, and speculatively pre-streams the sparse hint directly to Node A. Evaluating across three WAN regimes ($B \in \{15, 40, 80\}$ ms, $C$=15 ms; $N = 240$ queries/arm, zero transport errors) confirms consistent acceleration (Table IVb): cold TTFT drops by 16.9%–26.7%, warm TTFT drops by 24.5%–28.8%, and hit rates remain high ($h = 81.3\%$–$86.3\%$). SPHP completely flattens warm TTFT sensitivity to Link B RTT: baseline warm TTFT increases by only 9.9 ms across a 65 ms RTT increase (slope 0.153 ms/ms), proving multi-tier speculation immunizes users from multi-hop WAN delay.

### B. Live Unshaped Public Internet Validation (CM-4)

To verify that synthetic `netem` reflects production networks, we evaluated unshaped public internet routes ($N = 50$ queries/arm, 200 measured queries, `netem` disabled). On the two-node route (median TCP RTT 1.10 ms, p95 4.07 ms), P2-SPHP reduced cold TTFT from 1,730.72 ms to 1,308.60 ms (24.39% reduction, 28.0% median speedup; $h = 0.820$). On the three-node route originating from Node C (median RTT 4.33 ms, p95 10.46 ms), SPHP cut cold TTFT from 2,624.11 ms to 1,997.29 ms (23.89% reduction, 26.8% median speedup; $h = 0.840$), closely tracking synthetic emulation gains (23.7%–27.6%) under authentic internet jitter.

### C. Asymmetric Multi-Leg Stress Matrix (CM-5)

To evaluate SPHP under hostile topologies, we executed the full 27-cell CM-5 stress matrix ($B \in \{15, 40, 80\}$ ms with independent per-leg losses of 0.5%, 1.0%, and 2.0% on Link B, Link C, and both simultaneously; 54 arms, 810 queries, zero transport failures). SPHP-from-C achieved a mean TTFT reduction of 24.74% (peak 34.54%), maintaining a mean hit rate of 86.91% and document overlap of 0.624. Speedups remained uniform across tiers: 25.65% at $B = 15$ ms ($h = 85.93\%$), 22.23% at $B = 40$ ms ($h = 83.70\%$), and 26.33% at $B = 80$ ms ($h = 91.11\%$). Under concurrent 2% loss on both Link B and Link C at $B = 80$ ms, SPHP achieved a 34.54% TTFT reduction (2,812 → 1,841 ms) with 100% hit rate, proving robustness to multi-hop packet loss.

## IX. DISCUSSION & THREATS TO VALIDITY

### A. Systems Design Implications

Our findings contradict the intuition that WAN stage placement in distributed RAG is governed by network transmission delay. When identifier-sized boundaries are maintained across the network cut, WAN RTT is a second-order factor, whereas KV-cache state swings TTFT by up to 3.2×. Distributed RAG systems engineering should prioritize cache prefix warming and speculative prefill scheduling over wide-area link micro-optimizations.

### B. Client-Observed Goodput vs. Engine-Internal Metrics

Distinguishing client-observed goodput from server-internal engine throughput is critical in geo-distributed evaluation. Standard single-node deployments frequently report vLLM's internal Prometheus metrics (such as `avg_generation_throughput_tok_s` logged on `127.0.0.1`). In a distributed multi-tier architecture, relying on localhost engine metrics introduces three fundamental flaws: (1) *Network Transport Blindness:* Engine counters track only GPU CUDA forward passes on localhost, remaining blind to OS socket buffers (`SO_SNDBUF`, `tcp_wmem`), WireGuard cryptographic encapsulation, MTU fragmentation, WAN congestion pacing, and HTTP SSE stream framing at the edge gateway. Internal rates conceal transport stalls behind compute metrics. (2) *Continuous Batching Conflation:* Internal engine counters aggregate tokens across continuous batching iterations, conflating prompt prefill processing, KV-cache block allocation, and iteration scheduling with autoregressive decoding. (3) *Empirical Discovery of Transport Invariance:* Measuring client-observed goodput via Equation (1) established that decode throughput remains invariant at 35.2–35.5 tok/s across all regimes (0–80 ms RTT, clean to 2% loss). This proves that wide-area delay bottlenecks solely the first token; once streaming commences, token delivery operates at line rate without bubbles. Localhost logs could never validate this network transparency.

### C. Dynamic Overlap Threshold Heuristics

In SPHP, speculation commitment is governed by the set-overlap verification barrier $\text{Overlap}(\mathcal{H}, \mathcal{F}) \geq \theta$. While static $\theta = 0.5$ delivers robust acceleration ($h = 0.900$), workloads

with variable query clarity benefit from dynamic threshold calibration. We formalize an adaptive heuristic using BM25 score dispersion: $\Delta_{\mathrm{BM25}} = s_{\mathrm{BM25}}(d_1) - s_{\mathrm{BM25}}(d_{k_h})$, where $s_{\mathrm{BM25}}(d_i)$ is the lexical score of candidate $i$. Steep dispersion ($\Delta_{\mathrm{BM25}} \gg 0$) indicates high lexical confidence, enabling aggressive speculation ($\theta \to 0.4$); conversely, flat distributions reflect ambiguity, warranting conservative verification ($\theta \to 0.7$) to safeguard cloud GPU bandwidth.

### D. Limitations & Systems Generalizability

Three systems dimensions contextualize our findings. First, candidate agreement ($h = 0.900$) was characterized on MS MARCO; specialized corpora with divergent vocabulary distributions may exhibit different hit rates, requiring empirical threshold calibration. Second, our benchmarks evaluate sequential streams to isolate latency without multi-tenant GPU queueing; high-concurrency deployments should combine SPHP with prefill admission control and dynamic speculation throttling. Third, dense retrieval was profiled on an edge-class GPU (GTX 1660 Ti); while absolute stage durations scale with hardware compute, the core principles—the dominance of KV-cache residency over network delay and the uniform optimality of SPHP across WAN RTTs—generalize across distributed topologies.

### E. Artifact Availability & Reproducibility

All artifacts—including Tantivy/Qdrant indices, testbed harness, cost model scripts, raw confirmatory logs, and deterministic figure generation code—are available at https://github.com/ApatheticMioz/geo-distributed-hybrid-rag.

## X. Conclusion

We presented a comprehensive measurement study of stage-to-node placement for hybrid RAG across edge-cloud topologies. KV-cache state dominates latency variance (2.8–3.2×), whereas TTFT is essentially flat in WAN RTT (≈0.95 ms/ms). SPHP eliminates the prefill bottleneck by speculatively overlapping masked prefill with in-flight dense retrieval, cutting cold TTFT by 23.7%–27.6% ($p \leq 0.0008$, $h = 0.900$). Our validated analytical cost model confirms that SPHP split placement is globally optimal. Physical three-node tiering, live public WAN routes, edge failure recoveries, and multi-leg loss stress matrices demonstrate that speculative prefill acceleration provides robust, self-healing performance across distributed environments.

## References


[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9781–9793.

[2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "REALM: Retrieval-augmented language model pre-training," in *International Conference on Machine Learning (ICML)*, 2020, pp. 3979–3988.

[3] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. van den Driessche, J.-B. Lespiau, A. Lucier, F. Hutter *et al.*, "Improving language models by retrieving from trillions of tokens," in *International Conference on Machine Learning (ICML)*, 2022, pp. 2206–2240.

[4] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," 2024.

[5] S. Robertson and S. Walker, "Some simple effective approximations to the 2-poisson model for probabilistic weighted retrieval," in *Proceedings of the 17th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1994, pp. 232–241.

[6] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[7] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 2318–2335.

[8] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 135–142.

[9] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.

[10] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," in *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017, pp. 615–629.

[11] A. Banitalebi-Dehkordi, N. Vedula, J. Pei, F. Xia, L. Wang, and Y. Zhang, "Auto-Split: A general framework of collaborative edge-cloud AI," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining (KDD)*, 2021, pp. 2543–2553.

[12] S. Teerapittayanon, B. McDanel, and H.-T. Kung, "Distributed deep neural networks over the cloud, the edge and end devices," in *Proceedings of the 37th IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2017, pp. 1–11.

[13] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative LLM inference using phase splitting," in *Proceedings of the 51st ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2024.

[14] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024, pp. 193–210.

[15] K. B. Aruna, V. Kaliraj, I. Sudha, and V. Sureka, "Janus: Joint prefill/decode disaggregation with KV-cache-aware multi-cloud routing for edge-adjacent LLM serving," *EAI Endorsed Transactions on the Internet of Things*, 2025.

[16] Y. Bajaj, N. Craswell, M. Mitra, and M. Chen, "MS MARCO: A human-annotated open-domain dataset for machine reading comprehension," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 663–668.

[17] Y. Shen, M. Umar, S. Maeng, J. Suh, and I. Gupta, "Towards understanding systems trade-offs in RAG model inference," 2024.

[18] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the 40th International Conference on Machine Learning (ICML), PMLR 202*, 2023, pp. 19 274–19 286.

[19] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J.-J. Cappe, "Accelerating large language model decoding with speculative sampling," 2023.

[20] Y. Lyu, B. Irukulapati, and J. P. Champati, "Speculation at a distance: Where edge–cloud speculative decoding actually pays off," 2026.

[21] Z. Wang, Z. Wang, L. Le, H. S. Zheng, S. Mishra, V. Perot, Y. Zhang, A. Mattapalli, A. Taly, J. Shang, C. Lee, and T. Pfister, "Speculative RAG: Enhancing retrieval-augmented generation through drafting," in *International Conference on Learning Representations (ICLR)*, 2025.

[22] R. Fu, S. Li, Y. Xu, T. Wu, Z. Ma, and J. Shi, "Omnia: Efficient RAG serving through speculative scheduling," in *Proceedings of the 35th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '26)*, 2026.

[23] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot," in *Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST)*, 2025, pp. 155–170, best Paper.

[24] Y. Mei, Y. Zhuang, X. Miao, J. Yang, Z. Jia, and R. Vinayak, "Helix: Serving large language models over heterogeneous GPUs and network via max-flow," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025, pp. 586–602.

[25] J. A. Donenfeld, "WireGuard: Next generation kernel network tunnel," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2017.

[26] Qwen Team, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[27] W. Cheng, W. Zhang, H. Shen, Y. Cai, X. He, K. Lv, and Y. Liu, "Optimize weight rounding via signed gradient descent for the quantization of LLMs," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, autoRound / SignRound; INT4 (W4A16) via compressed-tensors.

[28] iproute2 project, "netem(8): Network emulator (tc qdisc)," iproute2 manual page, 2024, https://man7.org/linux/man-pages/man8/tc-netem.8.html.